

```
In [ ]: Algoritmos de Optimización | Actividad Guiada 1 (AG1)

Nombre: Jorge Ramilo Plata
URL Repositorio Github:
https://github.com/RamiPla/03MIAR_PRACTICAS/blob/main/Algoritmos
_AG1_Jorge%20Ramilo%20Plata.ipynb
```

A) EJECUTAR UNICAMENTE EL COMANDO

1. Torres de Hanoi con Divide y vencerás

```
In [8]: def Torres_Hanoi(N, desde, hasta):
        if N ==1 :
            print("Lleva la ficha " ,desde , " hasta " , hasta )

        else:
            #Torres_Hanoi(N-1, desde, 6-desde-hasta )
            Torres_Hanoi(N-1, desde, 6-desde-hasta )
            print("Lleva la ficha " ,desde , " hasta " , hasta )
            #Torres_Hanoi(N-1,6-desde-hasta, hasta )
            Torres_Hanoi(N-1, 6-desde-hasta , hasta )

Torres_Hanoi(3, 1 , 3)

Lleva la ficha 1 hasta 3
Lleva la ficha 1 hasta 2
Lleva la ficha 3 hasta 2
Lleva la ficha 1 hasta 3
Lleva la ficha 2 hasta 1
Lleva la ficha 2 hasta 3
Lleva la ficha 1 hasta 3
```

```
In [9]: #Sucesión de Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

```
Out[9]: 8
```

2. Devolución de cambio por técnica voraz

```
In [6]: def cambio_monedas(N, SM):  
        SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]  
        ValorAcumulado = 0  
  
        for i,valor in enumerate(SM):  
            monedas = (N-ValorAcumulado)//valor  
            SOLUCION[i] = monedas  
            ValorAcumulado = ValorAcumulado + monedas*valor  
  
        if ValorAcumulado == N:  
            return SOLUCION  
  
        cambio_monedas(15,[25,10,5,1])
```

```
Out[6]: [0, 1, 1, 0]
```

3. N-Reinas por técnica de vuelta atrás

```
In [5]: def escribe(S):
        n = len(S)
        for x in range(n):
            print("")
            for i in range(n):
                if S[i] == x+1:
                    print(" X ", end="")
                else:
                    print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos
    #reinas en la misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " +
        str(SOLUCION.count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

        #Verifica las diagonales
        for j in range(i+1, etapa +1 ):
            #print("Comprobando diagonal de " + str(i) + " y " +
            str(j))
            if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
        return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

reinas(8)
```

[1, 5, 8, 6, 3, 7, 2, 4]
[1, 6, 8, 3, 7, 4, 2, 5]
[1, 7, 4, 6, 8, 2, 5, 3]
[1, 7, 5, 8, 2, 4, 6, 3]
[2, 4, 6, 8, 3, 1, 7, 5]
[2, 5, 7, 1, 3, 8, 6, 4]
[2, 5, 7, 4, 1, 8, 6, 3]
[2, 6, 1, 7, 4, 8, 3, 5]
[2, 6, 8, 3, 1, 4, 7, 5]
[2, 7, 3, 6, 8, 5, 1, 4]
[2, 7, 5, 8, 1, 4, 6, 3]
[2, 8, 6, 1, 3, 5, 7, 4]
[3, 1, 7, 5, 8, 2, 4, 6]
[3, 5, 2, 8, 1, 7, 4, 6]
[3, 5, 2, 8, 6, 4, 7, 1]
[3, 5, 7, 1, 4, 2, 8, 6]
[3, 5, 8, 4, 1, 7, 2, 6]
[3, 6, 2, 5, 8, 1, 7, 4]
[3, 6, 2, 7, 1, 4, 8, 5]
[3, 6, 2, 7, 5, 1, 8, 4]
[3, 6, 4, 1, 8, 5, 7, 2]
[3, 6, 4, 2, 8, 5, 7, 1]
[3, 6, 8, 1, 4, 7, 5, 2]
[3, 6, 8, 1, 5, 7, 2, 4]
[3, 6, 8, 2, 4, 1, 7, 5]
[3, 7, 2, 8, 5, 1, 4, 6]
[3, 7, 2, 8, 6, 4, 1, 5]
[3, 8, 4, 7, 1, 6, 2, 5]
[4, 1, 5, 8, 2, 7, 3, 6]

[4, 1, 5, 8, 6, 3, 7, 2]
[4, 2, 5, 8, 6, 1, 3, 7]
[4, 2, 7, 3, 6, 8, 1, 5]
[4, 2, 7, 3, 6, 8, 5, 1]
[4, 2, 7, 5, 1, 8, 6, 3]
[4, 2, 8, 5, 7, 1, 3, 6]
[4, 2, 8, 6, 1, 3, 5, 7]
[4, 6, 1, 5, 2, 8, 3, 7]
[4, 6, 8, 2, 7, 1, 3, 5]
[4, 6, 8, 3, 1, 7, 5, 2]
[4, 7, 1, 8, 5, 2, 6, 3]
[4, 7, 3, 8, 2, 5, 1, 6]
[4, 7, 5, 2, 6, 1, 3, 8]
[4, 7, 5, 3, 1, 6, 8, 2]
[4, 8, 1, 3, 6, 2, 7, 5]
[4, 8, 1, 5, 7, 2, 6, 3]
[4, 8, 5, 3, 1, 7, 2, 6]
[5, 1, 4, 6, 8, 2, 7, 3]
[5, 1, 8, 4, 2, 7, 3, 6]
[5, 1, 8, 6, 3, 7, 2, 4]
[5, 2, 4, 6, 8, 3, 1, 7]
[5, 2, 4, 7, 3, 8, 6, 1]
[5, 2, 6, 1, 7, 4, 8, 3]
[5, 2, 8, 1, 4, 7, 3, 6]
[5, 3, 1, 6, 8, 2, 4, 7]
[5, 3, 1, 7, 2, 8, 6, 4]
[5, 3, 8, 4, 7, 1, 6, 2]
[5, 7, 1, 3, 8, 6, 4, 2]
[5, 7, 1, 4, 2, 8, 6, 3]

[5, 7, 2, 4, 8, 1, 3, 6]
[5, 7, 2, 6, 3, 1, 4, 8]
[5, 7, 2, 6, 3, 1, 8, 4]
[5, 7, 4, 1, 3, 8, 6, 2]
[5, 8, 4, 1, 3, 6, 2, 7]
[5, 8, 4, 1, 7, 2, 6, 3]
[6, 1, 5, 2, 8, 3, 7, 4]
[6, 2, 7, 1, 3, 5, 8, 4]
[6, 2, 7, 1, 4, 8, 5, 3]
[6, 3, 1, 7, 5, 8, 2, 4]
[6, 3, 1, 8, 4, 2, 7, 5]
[6, 3, 1, 8, 5, 2, 4, 7]
[6, 3, 5, 7, 1, 4, 2, 8]
[6, 3, 5, 8, 1, 4, 2, 7]
[6, 3, 7, 2, 4, 8, 1, 5]
[6, 3, 7, 2, 8, 5, 1, 4]
[6, 3, 7, 4, 1, 8, 2, 5]
[6, 4, 1, 5, 8, 2, 7, 3]
[6, 4, 2, 8, 5, 7, 1, 3]
[6, 4, 7, 1, 3, 5, 2, 8]
[6, 4, 7, 1, 8, 2, 5, 3]
[6, 8, 2, 4, 1, 7, 5, 3]
[7, 1, 3, 8, 6, 4, 2, 5]
[7, 2, 4, 1, 8, 5, 3, 6]
[7, 2, 6, 3, 1, 4, 8, 5]
[7, 3, 1, 6, 8, 5, 2, 4]
[7, 3, 8, 2, 5, 1, 6, 4]
[7, 4, 2, 5, 8, 1, 3, 6]
[7, 4, 2, 8, 6, 1, 3, 5]

[7, 5, 3, 1, 6, 8, 2, 4]

[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]

4. Viaje por el río. Programación dinámica

```
In [4]: TARIFAS = [
[0,5,4,3,999,999,999],
[999,0,999,2,3,999,11],
[999,999, 0,1,999,4,10],
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Iniciación de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(9999)*N]
    RUTA = [ [""]*N for i in range(9999)*N]

    for i in range(0,N-1):
        RUTA[i][i] = i                    #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0                 #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k          #Anota que para ir de i a j
hay que pasar por k
                    PRECIOS[i][j] = MIN

        return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
```

```

        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta]))
+ \
        ',' + \
        str(RUTA[desde][hasta] \
        )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

PRECIOS

```

[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

RUTA

```

[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']

```

La ruta es:

Out[4]: ',0,2,5'

B) MEJORAS EN EL algoritmo

1. Torres de Hanoi con Divide y vencerás

```
In [11]: def torres_hanoi(numero_discos, desde, hasta):
            if numero_discos <= 0:
                return 0

            if numero_discos == 1:
                print("Llevar la ficha desde", desde, "hasta", hasta)
                return 1

            auxiliar = 6 - desde - hasta
            movimientos = torres_hanoi(numero_discos - 1, desde,
auxiliar)
            print("Llevar la ficha desde", desde, "hasta", hasta)
            movimientos += 1
            movimientos += torres_hanoi(numero_discos - 1, auxiliar,
hasta)
            return movimientos

print("Ejemplo con 3 discos:")
total_movimientos = torres_hanoi(3, 1, 3)
print("Número total de movimientos:", total_movimientos)

assert total_movimientos == 7
assert torres_hanoi(0, 1, 3) == 0
print("Pruebas de Hanoi superadas.")
```

```
Ejemplo con 3 discos:
Llevar la ficha desde 1 hasta 3
Llevar la ficha desde 1 hasta 2
Llevar la ficha desde 3 hasta 2
Llevar la ficha desde 1 hasta 3
Llevar la ficha desde 2 hasta 1
Llevar la ficha desde 2 hasta 3
Llevar la ficha desde 1 hasta 3
Número total de movimientos: 7
Pruebas de Hanoi superadas.
```

\$\$ ***** NOTA ***** \$\$

```
In [ ]: El ejercicio se divide en tres pasos: el primer es mover los
discos pequeños a la torre auxiliar, el segundo sería mover el
disco grande al destino y por último volver a mover los discos
pequeños sobre él. y lo mismo hacemos con un disco menos, hasta
llegar al caso sencillo de un único disco.

La mejora que he realizado a sido dejar el algoritmo inicial y
lo que he hecho ha sido añadir el número de movimientos devuelto
por la función.
```

```
In [12]: def fibonacci(n):  
        if n < 2:  
            return 1  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print("Fibonacci(5):", fibonacci(5))  
assert fibonacci(5) == 8
```

Fibonacci(5): 8

2. Devolución de cambio por técnica voraz

```
In [14]: def cambio_monedas(cantidad, sistema):  
        solucion = [0] * len(sistema)  
        valor_acumulado = 0  
  
        for i, moneda in enumerate(sistema):  
            monedas = (cantidad - valor_acumulado) // moneda  
            solucion[i] = monedas  
            valor_acumulado += monedas * moneda  
  
            if valor_acumulado == cantidad:  
                return solucion  
  
        return None  
  
sistema_habitual = [25, 10, 5, 1]  
print("Cambio de 27 con", sistema_habitual, ":",  
      cambio_monedas(27, sistema_habitual))  
print("Cambio de 0:", cambio_monedas(0, sistema_habitual))  
print("Cantidad 3 con [4, 2]:", cambio_monedas(3, [4, 2]))  
  
assert cambio_monedas(27, sistema_habitual) == [1, 0, 0, 2]  
assert cambio_monedas(0, sistema_habitual) == [0, 0, 0, 0]  
assert cambio_monedas(3, [4, 2]) is None  
print("Pruebas del cambio voraz superadas.")
```

Cambio de 27 con [25, 10, 5, 1] : [1, 0, 0, 2]
Cambio de 0: [0, 0, 0, 0]
Cantidad 3 con [4, 2]: None
Pruebas del cambio voraz superadas.

\$\$ ***** NOTA ***** \$\$

En este ejercicio la función recorre las monedas de mayor a menor, para cada valor lo que hace es calcular cuantas monedas caben sin superar la cantidad, si el sistema no es capaz de formar el importe exacto nos devuelve el valor "None".

3. N-Reinas por técnica de vuelta atrás

```
In [16]: def es_prometedora(solucion, etapa):
    for i in range(etapa + 1):
        for j in range(i + 1, etapa + 1):
            misma_fila = solucion[i] == solucion[j]
            misma_diagonal = abs(i - j) == abs(solucion[i] -
solucion[j])

            if misma_fila or misma_diagonal:
                return False

    return True

def reinas(numero_reinas):
    soluciones = []
    solucion = [0] * numero_reinas

    def colocar_reina(etapa):
        for fila in range(1, numero_reinas + 1):
            solucion[etapa] = fila

            if es_prometedora(solucion, etapa):
                if etapa == numero_reinas - 1:
                    soluciones.append(solucion.copy())
                else:
                    colocar_reina(etapa + 1)

            solucion[etapa] = 0

    if numero_reinas > 0:
        colocar_reina(0)

    return soluciones

def mostrar_tablero(solucion):
    for fila in range(1, len(solucion) + 1):
        linea = []
        for columna in range(len(solucion)):
            if solucion[columna] == fila:
                linea.append("X")
            else:
                linea.append("-")
        print(" ".join(linea))

soluciones_4 = reinas(4)
soluciones_8 = reinas(8)

print("Soluciones para 4 reinas:", len(soluciones_4))
print(soluciones_4)
print("\nPrimer tablero de 4 reinas:")
mostrar_tablero(soluciones_4[0])
print("\nNúmero de soluciones para 8 reinas:",
```

```
len(soluciones_8))

assert len(reinas(1)) == 1
assert len(reinas(2)) == 0
assert len(soluciones_4) == 2
assert len(soluciones_8) == 92
print("Pruebas de N-Reinas superadas.")
```

Soluciones para 4 reinas: 2
[[2, 4, 1, 3], [3, 1, 4, 2]]

Primer tablero de 4 reinas:

```
- - X -
X - - -
- - - X
- X - -
```

Número de soluciones para 8 reinas: 92
Pruebas de N-Reinas superadas.

\$\$ ***** NOTA ***** \$\$

Como podemos ver, cada posición de la lista representa una columna y su valor indica la fila de la reina, se comprueba ante de nada que no haya dos filas iguales ni dos reinas en la misma diagonal, es resulta que una de las posiciones no es válida el algoritmo vuelvo hacia atrás y prueba otra fila.

Lo que he realizado en esta mejora de algoritmo ha sido eliminar el argumento mutable de la función inicial, guardar las soluciones y que se muestren sólo los resultados representativos.

4. Viaje por el río. Programación dinámica

```
In [19]: TARIFAS = [
    [0, 5, 4, 3, 999, 999, 999],
    [999, 0, 999, 2, 3, 999, 11],
    [999, 999, 0, 1, 999, 4, 10],
    [999, 999, 999, 0, 5, 6, 9],
    [999, 999, 999, 999, 0, 999, 4],
    [999, 999, 999, 999, 999, 0, 3],
    [999, 999, 999, 999, 999, 999, 0],
]

def calcular_precios(tarifas):
    numero_embarcaderos = len(tarifas)
    infinito = 9999
    precios = [[infinito] * numero_embarcaderos for _ in
range(numero_embarcaderos)]
    ruta = [[-1] * numero_embarcaderos for _ in
range(numero_embarcaderos)]

    for i in range(numero_embarcaderos):
        precios[i][i] = 0
        ruta[i][i] = i

    for i in range(numero_embarcaderos - 1):
        for j in range(i + 1, numero_embarcaderos):
            mejor_precio = tarifas[i][j]

            if mejor_precio < 999:
                ruta[i][j] = i
            else:
                mejor_precio = infinito

            for k in range(i + 1, j):
                if tarifas[k][j] < 999:
                    precio_candidato = precios[i][k] +
tarifas[k][j]

                    if precio_candidato < mejor_precio:
                        mejor_precio = precio_candidato
                        ruta[i][j] = k

            precios[i][j] = mejor_precio

    return precios, ruta

def reconstruir_ruta(ruta, desde, hasta):
    if desde == hasta:
        return [desde]

    anterior = ruta[desde][hasta]
    if anterior == -1:
        return []
```

```

    if anterior == desde:
        return [desde, hasta]

    return reconstruir_ruta(ruta, desde, anterior) + [hasta]

PRECIOS, RUTA = calcular_precios(TARIFAS)
ruta_0_6 = reconstruir_ruta(RUTA, 0, 6)

print("Tabla de precios mínimos:")
for fila in PRECIOS:
    print(fila)

print("\nTabla de rutas:")
for fila in RUTA:
    print(fila)

print("\nPrecio mínimo de 0 a 6:", PRECIOS[0][6])
print("Ruta completa de 0 a 6:", ruta_0_6)

assert PRECIOS[0][6] == 11
assert ruta_0_6 == [0, 2, 5, 6]
assert reconstruir_ruta(RUTA, 6, 0) == []
print("Pruebas del viaje por el río superadas.")

```

```

Tabla de precios mínimos:
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 9999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 9999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 0]

```

```

Tabla de rutas:
[0, 0, 0, 0, 1, 2, 5]
[-1, 1, -1, 1, 1, 3, 4]
[-1, -1, 2, 2, 3, 2, 5]
[-1, -1, -1, 3, 3, 3, 3]
[-1, -1, -1, -1, 4, -1, 4]
[-1, -1, -1, -1, -1, 5, 5]
[-1, -1, -1, -1, -1, -1, 6]

```

```

Precio mínimo de 0 a 6: 11
Ruta completa de 0 a 6: [0, 2, 5, 6]
Pruebas del viaje por el río superadas.

```

\$\$ ***** NOTA ***** \$\$

Lo que hace este código es que para cada pareja de embarcaderos se compara el viaje directo con las rutas que mediante un embarcadero intermedio y se reutilizan los mejores precios ya calculados.

La mejora que he llevado a cabo ha sido corregir el código original, ya que este aunque calculaba bien el precio, la función devolvía el valor 0,2,5 y omitía el destino 6, y con esta nueva función reconstruimos la ruta completa y nos dice cuando no existe un trayecto.